

Hermes configuration — field report

SUMMARY

The configuration was tuned to optimize the agent's cost / latency / reliability balance on our single-GPU infrastructure. Three areas of work: less aggressive context compression, safety caps on large inputs, and the addition of an anti-loop safeguard. Everything runs locally, with no dependency on an external service.

1 Context management

The context is the "memory budget" shared between the conversation history and the response. Beyond a certain fill level, Hermes automatically compresses it to stay within the model's limits.

compression.enabled: true

Enables automatic compression of the history. Without it, a long session would eventually exceed the context window.

compression.threshold: 0.80

Trigger threshold: compression kicks in when the history reaches 80% of the window. This later setting (raised from an initially earlier value) preserves the prefix cache for longer and smooths out latency spikes. It is the single highest benefit-to-effort adjustment.

compression.target_ratio: 0.30

Target after compression: the history is brought back down to 30% of the window. A deliberately moderate target, which limits information loss at each pass.

2 Input safeguards

file_read_max_chars: 30000

When reading a file, only the first 30,000 characters are ingested. Prevents a large file from saturating the context in one go.

tool_output.max_bytes: 30000 · tool_output.max_lines: 500

Same logic for tool output (shell command, search...), truncated at 30,000 bytes or 500 lines, whichever comes first. Prevents a verbose command from drowning the context. The two caps were aligned (30,000) to ensure consistent behavior between file reads and tool output.

These settings are safety caps: they protect against the scenario where a single large input would blow up the context — and the associated token consumption.

3 Memory and compression

`auxiliary.compression.provider: main`

Compression is handled by the main model itself (Qwen), not by a dedicated secondary model. On our single-GPU infrastructure, each compression therefore temporarily ties up the very model that serves responses. This effect is, however, greatly mitigated by the threshold raised to 80%: since compressions are now infrequent, their occasional impact on latency remains marginal. It is a deliberate trade-off in favor of simplicity (a single model to operate).

`memory.provider: holographic`

The agent's long-term memory system (what it retains between sessions). The "holographic" option is local and self-contained, with no external database (local SQLite) — a fitting choice for a deployment on our own machine.

4 Agent behavior

`agent.tool_use_enforcement: true`

Forces the agent to actually use its tools rather than settle for a text-only reply. Guarantees that it acts (commands, searches...) instead of merely describing. Consistent with an operational agent.

`agent.max_turns: 30`

Cap on the number of round-trips (reasoning + tool call + reading the result) the agent can chain on a task before stopping. Anti-loop safeguard: essential for execution safety and cost control, all the more so since the agent has terminal access.

`display.show_reasoning: false`

The model performs prior reasoning, but it is not shown to the user, who only sees the final answer. A deliberate interface choice. Note that the cost of this reasoning (time and tokens) is still paid; it is simply hidden.

5 Slack integration

`platforms.slack.busy_ack_detail: true`

When the agent is busy on Slack, it acknowledges by specifying what it is currently doing, rather than a plain "working...". Improves visibility and user experience.